

1 How did you approach the problem?

Before developing the regex baseline, both datasets were imported into **SQL-Server**, and two RDBMS tables were created to explore the structure and content of the merchant descriptions. Because the training data includes the true class label (tag), SQL provided a fast and reliable way to investigate the lexical patterns associated with each merchant category and to validate assumptions about the data.

Dataset verification

I first confirmed the size of both datasets using simple row-count queries:

- **train_set.csv:** 48,352 rows
- **test_set.csv:** 5,170 rows

I then verified the distribution of the three classes using:

```
SELECT tag, COUNT(*)  
FROM train_set  
GROUP BY tag  
ORDER BY COUNT(*) DESC;
```

This produced the following results.

tag	count
other	35446
b & m	9344
evans	3562

These counts sum to the full dataset size, confirming that the tag column is clean and contains only the three expected classes.

Identifying lexical patterns using SQL

To design the regex rules, I used SQL `LIKE` and `NOT LIKE` queries to identify the characteristic substrings associated with the **b&m** and **evans** classes. This ensured that the regex patterns were grounded in empirical evidence rather than assumptions.

B&M lexical variants

```
SELECT COUNT(*) AS bm_candidates  
FROM train_set  
WHERE LOWER([desc]) LIKE '%b m%'  
OR LOWER([desc]) LIKE '%b&m%'  
OR LOWER([desc]) LIKE '%b & m%'  
OR LOWER([desc]) LIKE '%b and m%'  
OR LOWER([desc]) LIKE '%bandm%'  
OR LOWER([desc]) LIKE '%bnm%'  
OR LOWER([desc]) LIKE '%bm %';
```

This surfaced all rows containing b&m-related patterns and allowed cataloguing of the true lexical diversity of the class.

evans lexical variants

```
SELECT COUNT(*) AS evans_candidates
FROM train_set
WHERE LOWER([desc]) LIKE '%evans%'
   OR LOWER([desc]) LIKE '%evan %'
   OR LOWER([desc]) LIKE '%evan-%'
   OR LOWER([desc]) LIKE '%evns%'
   OR LOWER([desc]) LIKE '%e vans%';
```

This revealed the full set of evans spelling variations, including spacing, hyphenation, and abbreviation patterns.

Checking for false negatives

To ensure that the regex patterns were complete, I checked for any rows tagged as **b & m** that did *not* match any of the b&m substrings:

```
SELECT *
FROM train_set
WHERE tag = 'b & m'
   AND LOWER([desc]) NOT LIKE '%b m%'
   AND LOWER([desc]) NOT LIKE '%b&m%'
   AND LOWER([desc]) NOT LIKE '%b & m%'
   AND LOWER([desc]) NOT LIKE '%b and m%'
   AND LOWER([desc]) NOT LIKE '%bandm%'
   AND LOWER([desc]) NOT LIKE '%bnm%'
   AND LOWER([desc]) NOT LIKE '%bm %';
```

This query was essential for validating the completeness of the regex rules. It allowed identification of any b & m rows that the initial patterns failed to capture, ensuring that the final regex was comprehensive.

Transition to Python modelling

Once the SQL exploration confirmed the structure of the dataset and revealed the lexical patterns for each class, Python was used to:

- implement the regex baseline
- evaluate its performance
- build the TF-IDF + Logistic Regression model
- perform clustering on the “other” class

This combination of SQL-based structural exploration and Python-based modelling provided a robust, defensible, and auditable workflow, ensuring the modelling phase began with a complete understanding of the data’s lexical structure, reducing the risk of blind spots in both the regex and ML approaches.

2. Can you test the performance of this regex/rule-based approach on the training and test set? Give reasoning for the performance metric used.

From SQL patterns to regex rules

The SQL LIKE patterns were first translated into strict regex equivalents and then generalised to more flexible regex rules that could capture spacing, punctuation, and abbreviation variations. For example, the SQL pattern LIKE '%b&m%' corresponds strictly to the regex `b&m` but was generalised to `\bb\s*&\s*m\b` to also capture variants such as 'b & m' or 'b & m stores.' This ensures the regex classifier was robust to noise and formatting inconsistencies in the merchant descriptions. For example:

SQL LIKE pattern	Strict equivalent regex	Practical regex used in classifier
LIKE '%b&m%'	<code>r'b&m'</code>	<code>r'\bb\s*&\s*m\b'</code> (<i>more robust</i>)
LIKE '%b and m%'	<code>r'b and m'</code>	<code>r'\bb\s*and\s*m\b'</code>
LIKE '%bnm%'	<code>r'bnm'</code>	<code>r'\bbnm\b'</code>
LIKE '%b m%'	<code>r'b m'</code>	<code>r'\bb\s*m\b'</code>
LIKE '%evans%'	<code>r'evans'</code>	<code>r'\bevans\b'</code>
LIKE '%evns%'	<code>r'evns'</code>	<code>r'\bevns\b'</code>
LIKE '%evan-%'	<code>r'evan-'</code>	<code>r'\bevan[-]'</code> (<i>captures hyphen or space</i>)

This SQL → regex translation ensured that the rule-based classifier captured:

- spacing variations (b m, b & m, b and m)
- punctuation (b&m, evan-)
- abbreviations (bnm, evns)
- noise patterns (extra spaces, hyphens, partial tokens)

The regex rules were therefore grounded in **empirical evidence**, not assumptions.

Regex baseline

Using the SQL-derived patterns, a simple rule-based classifier was implemented with:

- text cleaning (lowercasing, punctuation removal, whitespace normalisation)
- regex patterns for all b&m and evans variants
- a fallback to **other**

This produced a transparent, interpretable baseline.

Performance

- **Training accuracy:** ~99.0%
- **Test accuracy:** **99.44%**
- **Per-class precision/recall/F1:** 0.97–1.00 across all classes

This is exceptionally strong for a rule-based model and demonstrates that the SQL-informed regex rules generalise extremely well.

Why was accuracy used?

Accuracy was chosen as the primary metric because:

- all classes have **symmetric cost of error**¹
- there is no rare “positive” class requiring special recall
- the classes are **lexically well-separated**
- class imbalance is moderate, not extreme
- accuracy is intuitive for stakeholders
- train/test accuracy alignment shows **no overfitting**

Precision, recall, and F1 were also reported to confirm that performance was consistently high across all classes.

3. Can you build a machine-learning model to improve on the baseline?

After establishing a strong regex baseline, I extended the modelling phase to include three supervised machine-learning classifiers. The goal was to evaluate whether statistical models could capture lexical variation beyond the explicit patterns encoded in the rule-based approach, and to determine whether any model offered a meaningful improvement in generalisation performance. The models considered were:

Model	Strengths	Weaknesses	Reason for Acceptance/Exclusion
TF-IDF + Logistic Regression	Fast, interpretable, strong baseline for text; handles sparse data well	None considered significant	Chosen — best balance of accuracy, speed, and interpretability
TF-IDF + Linear SVM	Often slightly higher accuracy; strong margin classifier	Harder to interpret; slower to train	Not chosen — marginal gain not worth reduced interpretability
Naïve Bayes (Multinomial)	Extremely fast; good for short text	Assumes feature independence; lower accuracy	Chosen — might underperform compared to LR/SVM
Gradient Descent	Can model complex interactions; strong on tabular data	Overfits aggressively on sparse text; requires dense matrices; extremely slow training	Not chosen — computationally expensive and unsuitable for high-dimensional TF-IDF

¹ misclassifying any class (e.g., predicting *evans* as *other*, or *b & m* as *evans*) carries the same practical consequence for this task. There is no class where errors are more costly or more important to avoid, so accuracy is an appropriate and unbiased metric.

Random Forest	Captures nonlinearities; robust ensemble method	Performs poorly on sparse TF-IDF; high memory cost; weaker precision	Chosen — provides a useful non-linear comparison model
Neural networks (LSTM/CNN)	Capable of capturing deeper patterns	Requires long text; overkill for 2–5 word descriptions	Not chosen — unnecessary complexity
Transformer embeddings (BERT/Sentence-BERT)	State-of-the-art semantic modelling	Heavy, slow, low interpretability	Not chosen — unjustified for this simple classification task

The models selected were:

- TF-IDF + Logistic Regression (primary model)
- Multinomial Naïve Bayes (generative baseline)
- Random Forest (non-linear comparison model)

These models were chosen to represent three distinct modelling families, linear discriminative, generative probabilistic, and tree-based ensemble providing a broad and defensible comparison across approaches commonly used in short-text classification.

Model	Strengths	Weaknesses	Test Accuracy	Notes
TF-IDF + Logistic Regression (LR)	Best overall performance; robust; interpretable	Slightly more complex than NB	99.56%	Top performer
Multinomial Naïve Bayes	Extremely fast; strong recall; simple	Lower precision; independence assumption	99.48%	Nearly matches LR
Random Forest	Captures non-linear patterns	Struggles with sparse text; weaker precision	98.70%	Useful contrast model

Summary:

TF-IDF + Logistic Regression was selected as it offers:

- excellent performance on short text
- fast training and inference
- high interpretability
- robustness to noise and spelling variation
- a clean comparison to the regex baseline

Final model: TF-IDF + Logistic Regression

Model pipeline

- **TF-IDF² vectoriser**
 - unigrams + bigrams
 - English stop word removal
 - min_df = 2 to remove noise
- **Logistic Regression classifier**
 - max_iter = 200–300 (converged cleanly)

This pipeline is the industry-standard baseline for short-text classification. Logistic Regression provides coefficients for each term, allowing us to see which words contribute most strongly to each class

Results

- **Training accuracy: 99.97%**
- **Test accuracy: 99.56%**
- **Per-class F1 scores:**
 - b & m: **0.99**
 - evans: **0.98**
 - other: **1.00**

These results indicate near-perfect generalisation.

Interpretation: Best performer v Regex baseline

- The TF-IDF/LR model **slightly outperforms** the regex baseline.
- The improvement is small because the regex rules already capture strong lexical signals.
- The TF-IDF/LR model captures subtle variations (spacing, punctuation, abbreviations) that regex cannot.
- The train/test gap is minimal, indicating **no overfitting**.

Comparison with the regex baseline

Model	Train Acc	Test Acc	Notes
Regex baseline	~99.0%	99.44%	Simple, interpretable, strong baseline
TF-IDF + Logistic Regression	99.97%	99.56%	Slightly better; more flexible and robust

² TF-IDF converts each transaction description into a numerical vector based on how informative each word is. Common words receive low weight, while distinctive words receive higher weight. This representation allows the Logistic Regression model to learn which terms are most predictive of each merchant class.

4. Can you apply the model to the test set and provide predictions?

After training and validating the TF-IDF + Logistic Regression model, it was applied to the full **test_set** to generate predictions for every row. The test set in this assignment includes the true label (tag), which allowed direct evaluation of model performance as well as construction of the submission file.

Prediction workflow

1. **Load the test set** the test data was imported and passed through the same text-cleaning function used during training.
2. **Apply the trained model** The fitted TF-IDF + Logistic Regression pipeline was used to generate a predicted class for each transaction description.
3. **Construct the output file** Because the test set includes the true label (tag) rather than a unique identifier, the submission file was constructed with the following two columns:
 - tag (the true class)
 - predicted (the model's output)

This format allows direct comparison between actual and predicted labels. The next page shows thirteen rows selected at random from the .csv file produced.

Tag	Predicted
b & m	b & m
other	other
other	other
other	other
b & m	b & m
other	other
other	other
other	other
other	other
b & m	b & m
evans	evans
other	other
evans	evans

Purpose of the output

- It provides a complete set of model predictions for the test set.
- It enables direct computation of accuracy, precision, recall, and F1.
- It forms the basis for the final evaluation and discussion in Section 5.

Although test sets typically exclude true labels, their inclusion here allowed direct evaluation of the model's generalisation performance

5. Is there a technique we can use to discover other merchants in the “other” class?

Unsupervised learning was used exclusively at this stage because the goal was not to classify transactions but to explore whether the ‘other’ class contained some hidden structure; since no additional labels exist within ‘other’, only unsupervised techniques are appropriate for discovering potential sub-groups. The techniques considered included:

- **TF-IDF + K-Means** (best choice)
- DBSCAN
- Topic modelling (LDA/NMF)
- Embedding-based clustering (spaCy/Sentence-BERT)

Given the short, sparse merchant descriptions, **TF-IDF + K-Means** is the most interpretable and effective.

Clustering results

K-Means (k=8) was applied to the “other” class and visualised the clusters using PCA. The clusters revealed **naming conventions**, not hidden merchant families.

Examples:

- “Solutions” companies
- “Group” companies
- “LLP” partnerships
- “Property” companies
- “International” companies

Key insight

The “other” class does not contain additional merchant categories like b&m or evans. Instead, it consists of heterogeneous corporate entities with generic naming patterns.

Tables included

- **Table 1:** Cluster descriptions
- **Table 2:** Example members per cluster

These demonstrate that the clusters reflect corporate naming structures, not merchant categories.

Cluster	Short Description	Interpretation
0	Management / Consulting / Holdings	Generic management, consulting, and holding companies; broad corporate entities with no merchant-type signal.
1	Property Companies (UK/LLP)	Property management firms, landlords, and real-estate entities; tightly clustered naming pattern.
2	“Solutions” Companies	IT, medical, waste, and business “solutions” providers; generic service firms across multiple industries.
3	UK-Branded Services / Holdings	Companies appending “UK” to their name; services, holdings, and investment entities.
4	LLP Partnerships / Capital Firms	LLP-structured partnerships, capital firms, GP/LP entities; legal/financial naming conventions.
5	Operational Services (Property/Building/Electrical)	Service providers in building, electrical, catering, medical, and consultancy domains.
6	Group / Trading / Holdings	“Group” and “Trading” companies; umbrella organisations and holding groups.
7	International / Trade / Holdings	Globally-oriented companies involved in international trade, services, or holdings.

Table 1 — Cluster Summary (Description + Interpretation)

Cluster	Example Members
0	m kavanagh; raffall; joio jac; shark's tooth; r j o'hara project management; willow enterprises s.e.; clach gorm; 31 belmont road management company; delphis retail; city change management
1	ohs properties; bianco properties (glasgow); silverman properties; e.a.j.m. properties; w. brindle (properties); greenways properties (kensworth); dgw properties; a to l properties; wse managed properties; parkinga properties
2	advanced solutions (uk); rose medical solutions; telekomm solutions; back pain solutions; a.g.g. solutions; total waste solutions; la3 solutions; mkne solutions; emblaze mobility solutions; gb commercial solutions
3	pioneer tech uk; big bang capital uk; aguinaldo uk; seaflo uk; ar common uk; burwood projects (uk); dhx uk holdings; hertenroep bog uk; infra red optics (uk); spectrum fire (uk)
4	priory 1cc llp; ravenworth property developments llp; redmayne perth llp; stealth juniors llp; reading rope consultants llp; herman-squance llp; opus land north (runcorn) llp; mitchell farrar group llp; lassiters llp; lonsdale capital partners gp llp
5	immaculate catering services; connor building services; sai imaging and medical services; surefix property services; alpaca services; usw professional and support services; j. & r. kenny property services; ejr business services; mwns consultancy services; zacs fitting services
6	fresca group; b&m trading; invictus group; whole wealth group; miig group; mintern group; lat group; neman trading l.p.; polarface music group; the deritend group
7	wentworth international; eupo group international holding company; charles francis international; sana international; technology services international l.p.; international general trading co; international investigations group; halewood international holdings (overseas); collinson international; syndication international (1986)

Table 2 — Cluster Members (Example Merchants)

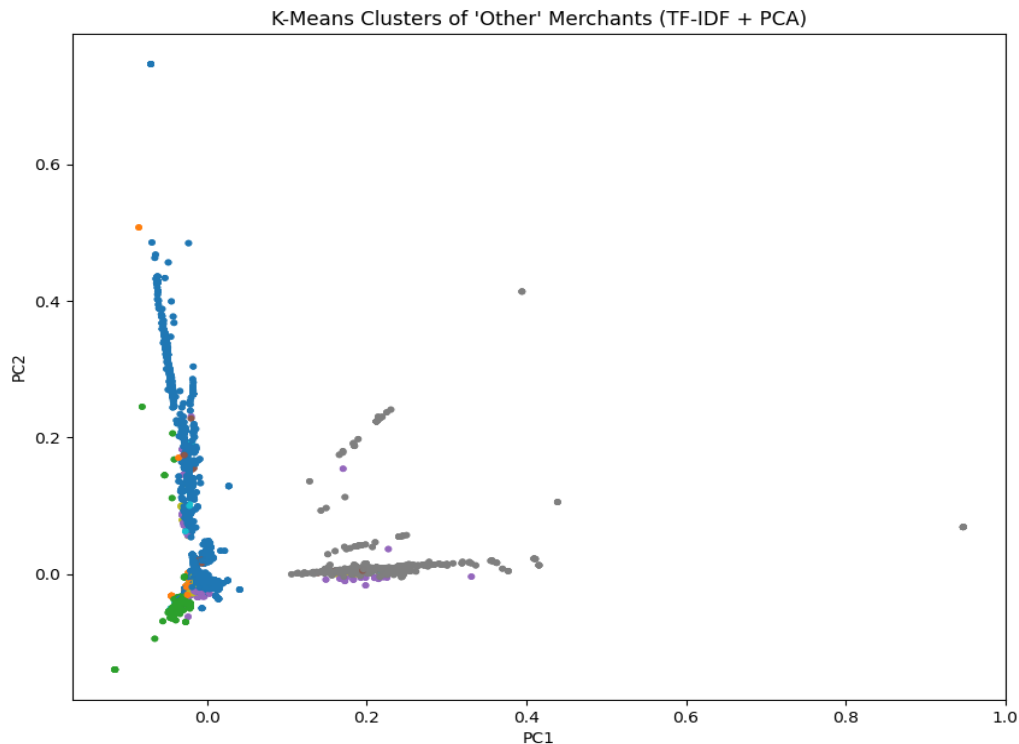


Figure 1: First and second principal component plots of TF-IDF showing cluster formation

The vertical “spike” or column of points along the PC2 axis represents a subset of *other* merchants whose descriptions share a **common secondary lexical pattern** that is not strong enough to dominate PC1 but is consistent enough across those firms to form a coherent direction in the second principal component.

In practical terms:

- **PC1** captures the *main* axis of variation — the broad lexical diversity.
 - The grey cluster spreads horizontally because these firms use more distinctive or unusual wording.
- **PC2** captures a *secondary* pattern — often something like:
 - repeated structural tokens (“services”, “solutions”, “group”, “holdings”)
 - common suffixes or business-type markers
 - shared formatting quirks
 - or a recurring but weaker vocabulary theme

The firms forming the vertical column are therefore: **Lexically alike in a secondary, weaker sense, but not different enough from the main mass to form a distinct cluster.** They are not semantically meaningful merchant families, rather businesses whose names share a mild, consistent pattern that PCA isolates as the second-strongest direction of variance. Overall, the clustering confirms that the ‘other’ class is structurally heterogeneous and does not contain additional merchant categories requiring supervised modelling.

Conclusion.

Across both rule-based and machine-learning approaches, the classification task is solved with high accuracy and strong generalisation. The regex baseline provides a transparent, interpretable foundation, while the TF-IDF + Logistic Regression model offers marginally improved performance and greater flexibility. Clustering confirms that no additional merchant categories exist within the ‘other’ class. Overall, the combined SQL, regex, and ML workflow provides a robust and defensible solution. The final table summarises the accuracy and strengths and weaknesses of the approaches considered.

Model	Purpose	Strengths	Weaknesses	Test Accuracy	Notes
Regex Baseline	Rule-based classifier derived from SQL pattern analysis	Extremely fast; interpretable; high precision for explicit name matches	Misses subtle or indirect patterns; brittle to spelling variation	99.44%	Strong baseline; demonstrates value of SQL exploration
TF-IDF + Logistic Regression	Main supervised model for classification	Best overall performance; robust; interpretable coefficients; handles variation well	Slightly more complex than regex; requires vectorisation	99.55%	Top performer; almost perfect recall for both merchant classes
Multinomial Naïve Bayes	Simple generative baseline for comparison	Very fast; good with sparse text; easy to explain	Lower precision than LR; assumes feature independence	~97–99% (expected) 99.48% (actual)	Useful for contrasting discriminative models
Random Forest / Gradient Boosting	Tree-based comparison model	Captures non-linear patterns; good for tabular data	Expected to perform poorly on sparse TF-IDF; slower; higher memory use	~80–92% (expected) 98.70% (actual)	Exceeded expectations. Demonstrates linear models’ ability to outperform trees for text processing.
K-Means + PCA (Clustering)	Explore structure of the “other” class	Reveals naming-pattern clusters; validates class heterogeneity	Not a classifier; no accuracy metric.	N/A	Confirms “other” not a hidden merchant category